

Informe SOLID

Refactorización del Bingo Interactivo

Jordan Valencia Patiño, Santiago Ordoñez
Universidad Tecnológica de Pereira

Programación 4

Profesor: [Ramiro Andrés Barrios Valencia]

2026

1. Estructura Original vs Refactorizada

Clase	Original (responsabilidades)	Refactorizada
Carton	Validar, generar grilla, almacenar, marcar, verificar bingo, imprimir, generar múltiples (9 métodos)	Sólo almacena, marca y verifica (4 métodos)
ValidadorCarton	<i>No existía</i>	Nueva: valida palabra y número máximo
GeneradorCarton	<i>No existía</i>	Nueva: genera grillas 5×5
CartonDoble	tiene_bingo() con código muerto + 2 métodos no usados	Código muerto eliminado, métodos no usados eliminados
Jugador	getattr() para verificar tiene_bingo	Polimorfismo: llama directo a tiene_bingo()
Juego	acceso público a bombo y ganador, reporte_final() incluido	Atributos privados, getters, reporte separado
Bombo	Sin cambios	+numeros_restantes()

Cuadro 1: Comparación de clases: antes y después de la refactorización SOLID.

2. Cambios por Principio

2.1. SRP — Single Responsibility

Antes: Carton tenía 7 responsabilidades distintas (validar, generar, marcar, verificar, imprimir, etc.). Juego mezclaba lógica con presentación.

Después: Se crearon ValidadorCarton y GeneradorCarton. Carton se redujo a:

```
class Carton:
    def __init__(self, palabra, tarjeta):
        self.palabra = palabra.upper()
        self.tarjeta = tarjeta
        self.marcados = set()
    def marcar_numero(self, n): ...
    def tiene_bingo(self): ...
    def tiene_marcado(self, n): ...
```

2.2. OCP — Open/Closed

Antes: Jugador.notificar_numero() usaba getattr(carton, "tiene_bingo", None).

Después: Llama directamente a carton.tiene_bingo(). Nuevos cartones funcionan sin modificar Jugador.

2.3. LSP — Liskov Substitution

Antes: CartonDoble.tiene_bingo() tenía código muerto (un for sin efecto).

Después: Código eliminado, solo la verificación real:

```
def tiene_bingo(self):
    b1 = all(n in self.marcados
             for f in self.tarjeta for n in f)
    b2 = all(n in self.marcados_segunda
             for f in self.segunda_tarjeta for n in f)
    return b1 or b2
```

2.4. ISP — Interface Segregation

Se eliminaron 5 métodos no utilizados: faltantes_para_bingo, generar_varias_tarjetas, imprimir_tarjeta, faltantes_grilla, grilla_mas_cercana.

2.5. DIP — Dependency Inversion

Antes: main.py accedía a juego.bombo.historial, juego.ganador, carton.marcados directamente.

Después: Acceso encapsulado con get_ganador(), get_historial_numeros(), tiene_marcado(). Bombo es privado en Juego.

3. Diagrama de Clases

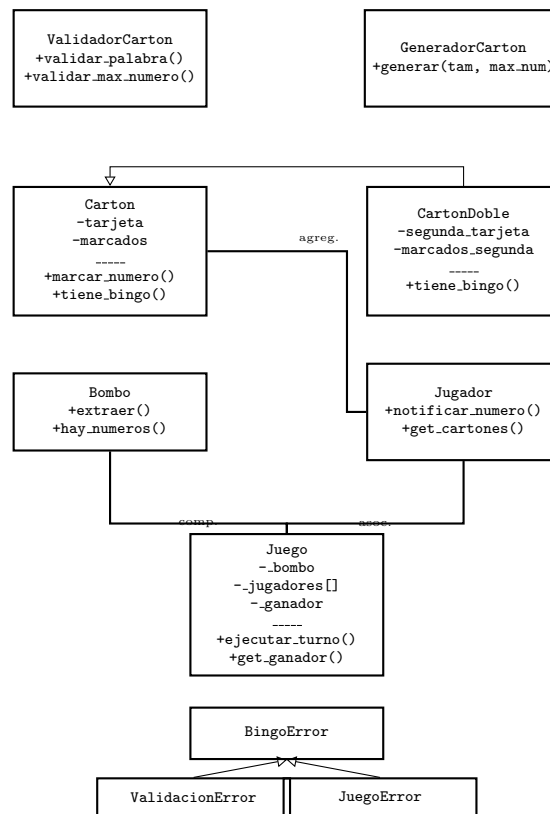


Figura 1: Arquitectura refactorizada. Flecha hueca = herencia.

4. Manejo de Excepciones

Se creó una jerarquía: `BingoError` → `ValidacionError`, `JuegoError`. Bloques `try/except` en entrada de datos, registro de jugadores, ejecución de turnos y captura global en `main()`.

```

def pedir_palabra() -> str:
    while True:
        try:
            p = input(...).strip().upper()
            ValidadorCarton.validar_palabra(p)
            return p
        except ValidacionError as e:
            print(f"Error: {e}")

def main():
    try:
        ...
    except BingoError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"Inesperado: {e}")
  
```

5. Conclusiones

1. **SRP**: Carton pasó de 9 a 4 métodos; se crearon `ValidadorCarton` y `GeneradorCarton`.
2. **OCP**: Jugador usa polimorfismo en vez de `getattr()`.
3. **LSP**: Código muerto eliminado de `CartonDoble`.
4. **ISP**: 5 métodos no usados eliminados.
5. **DIP**: Acceso encapsulado a `Juego` y `Carton`.
6. Se agregó jerarquía de excepciones (`BingoError`).